



基于微服务架构的智能物流管理系统

汇报人：付苗
指导老师：牛志华
汇报时间：2026.03.27

目录

01 选题背景和意义

02 需求分析

03 系统设计

04 功能实现

05 项目亮点

06 功能展示

07 后续计划

Part One

选题背景和意义



Part Two

需求分析



管理员 Admin

- 检测系统整体情况
- 管理所有用户账号
- 监控全部订单状态
- 管理仓库信息

商家 Shop

- 管理店铺信息与商品
- 管理仓库与库存
- 处理订单发货
- 查看店铺营收数据（后续开发）



顾客 Customer

- 浏览商城商品、加入购物车
- 下单、支付、查看订单进度
- 查看配送路径
- 管理收货地址

司机 Driver

- 接收配送任务
- 实时上报 GPS 位置
- 查看系统规划路线
- 管理绑定车辆信息

02

Part Two 需求分析-业务流程

顾客下单

商家发货

司机接单

确认送达



系统内生成订单信息
订单状态：待支付/代发货 - 0/1

根据商家选择发货仓库，生成物流路线
订单状态：待揽件

司机实时上报轨迹路线，订单中可查询
订单状态：派送中

订单状态：已完成



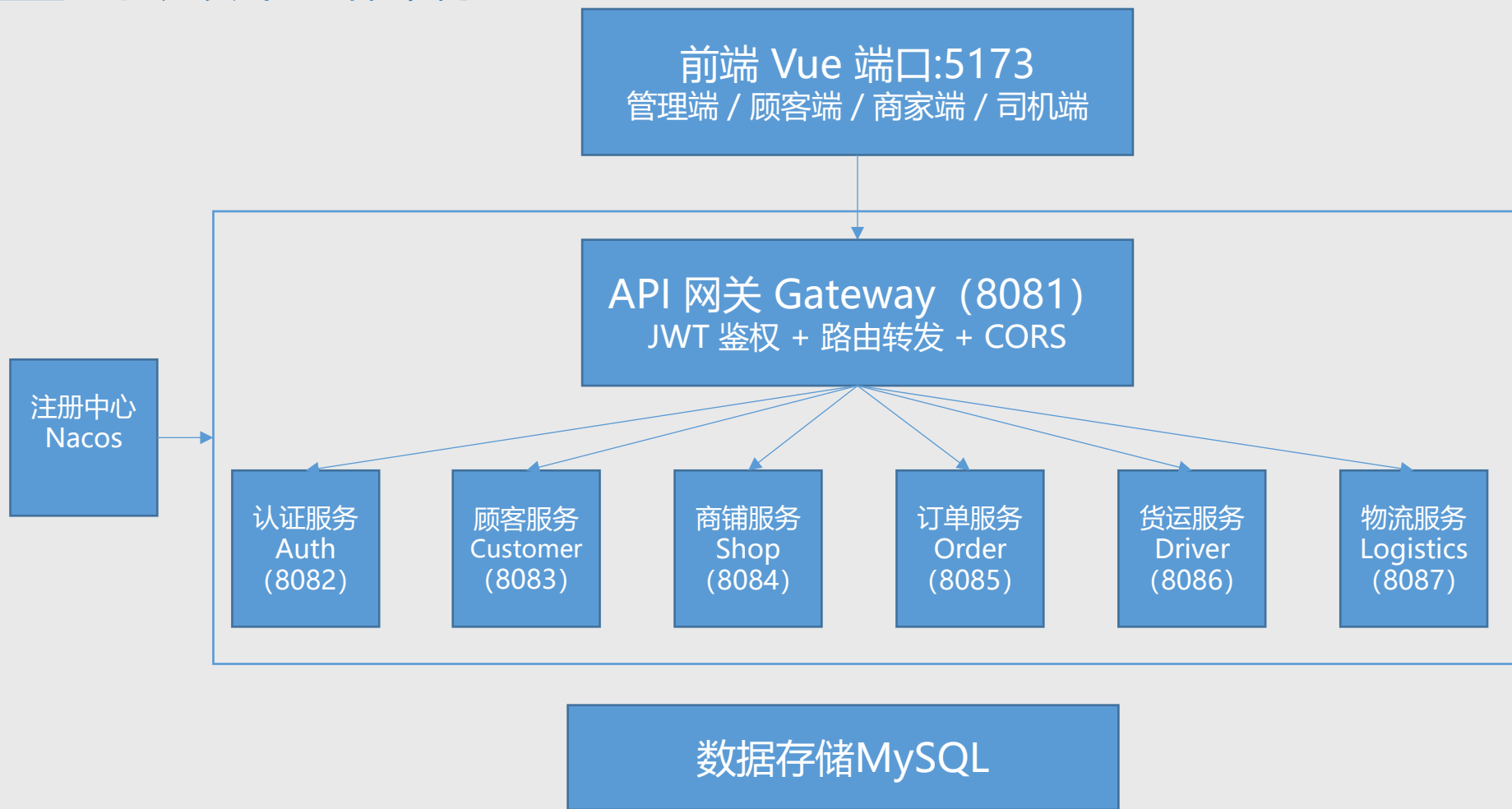
Part Three

系统设计



03

Part Three 系统设计-整体架构



当前服务间通信使用OpenFeign，预计 后续扩展为RabbitMQ，并组合使用Redis做系统缓存。

后端

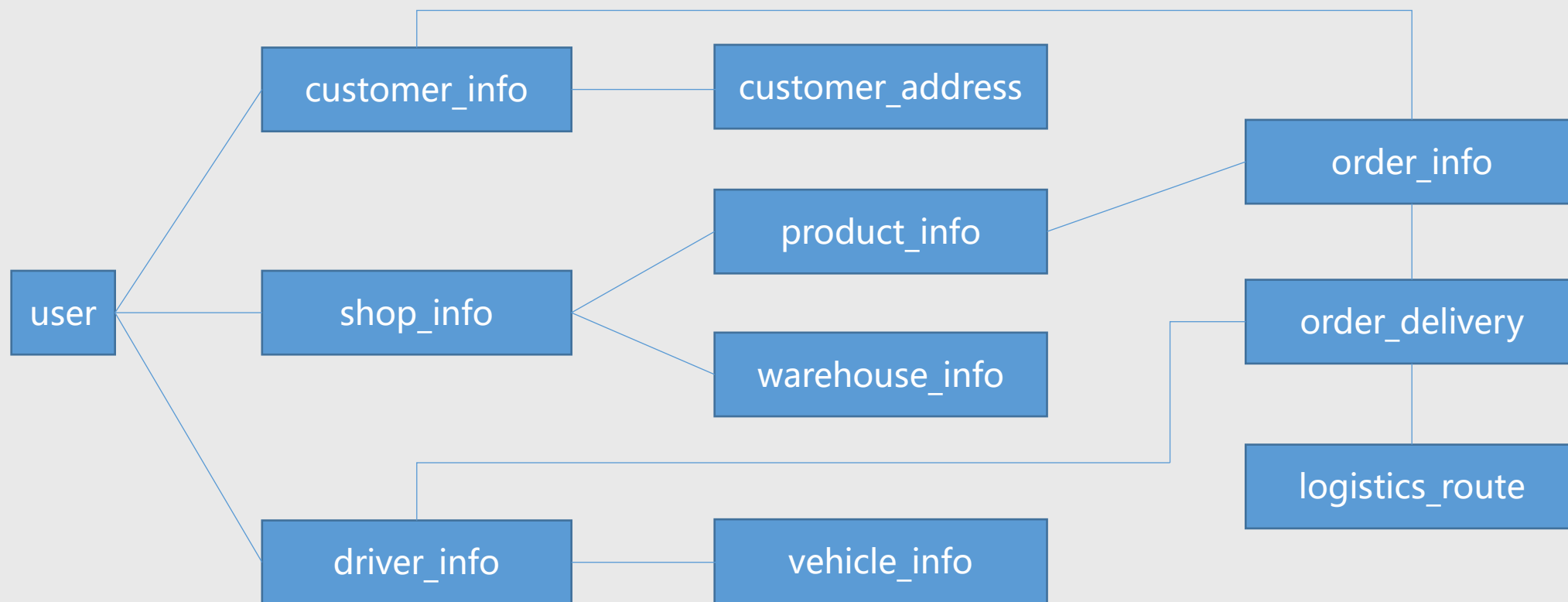
技术	用途
Spring Boot	服务基础框架
Spring Cloud	微服务治理
Spring Cloud Alibaba	Nacos集成
MyBatis Plus	ORM数据访问
OpenFeign	服务间调用
GraphHopper	路图信息读取
SpringDoc OpenAPI	接口文档
JWT	JWT令牌

前端

技术	用途
Vue3 + Vite8	前端框架
TypeScript	类型安全
Element Plus	UI组件
Pinia	状态管理
Axios	请求处理
Leaflet	地图渲染
Vue Router	路由管理

03

Part Three 系统设计-数据库设计



Part Four

功能实现



登录认证（端口8082）

- 用户登录接口：`POST /api/auth/login`
- 密码使用 BCrypt 哈希存储
- 登录成功后签发 JWT Token，有效期 2 小时

Token 中携带：

userId、username、permission（权限信息）

网关服务（端口8083）

- 所有请求经网关统一入口
- 白名单放行：`/api/auth/login`
- 读取 Token 中的 permission 信息，与请求路径的角色要求匹配
- 通过后将用户信息注入请求头，下游服务直接读取

浏览器发起请求

网关JWT验证

业务服务

顾客服务 (端口 8083)

- 顾客信息管理
增删改查用户信息 (customer_info)
为管理员提供用户管理接口
- 收货地址管理
- 内部 Feign 接口供 order-service 调用

```
public interface CustomerService {  
    /** 根据user_id获取顾客信息 */  
    Customer getCustomerByUserId(Long userId);  
  
    /** 根据customer_id获取顾客信息 */  
    Customer getCustomerById(Long customerId);  
  
    /** 创建或更新顾客信息 */  
    Customer saveOrUpdateCustomer(Customer customer);  
  
    /** 获取所有顾客列表 (管理员使用) */  
    List<Customer> getAllCustomers();  
  
    /** 更新顾客状态 */  
    boolean updateCustomerStatus(Long customerId, Integer status);  
  
    /** 删除顾客信息 */  
    boolean deleteCustomer(Long customerId);  
}
```

顾客服务 (端口 8083)

- 顾客信息管理

- 收货地址管理

增删改查用户地址，以customer_id为外键

前端返回具体地址，通过高德API接口转为经纬度存放在数据库中

- 内部 Feign 接口供 order-service 调用

```
public interface AddressService {  
    /** 根据customer_id获取所有地址 */  
    List<Address> getAddressesByCustomerId(Long customerId);  
  
    /** 根据address_id获取地址 */  
    Address getAddressById(Long addressId);  
  
    /** 添加地址 */  
    Address addAddress(Address address);  
  
    /** 更新地址 */  
    Address updateAddress(Address address);  
  
    /** 删除地址 */  
    boolean deleteAddress(Long addressId);  
  
    /** 设置默认地址 */  
    void setDefaultAddress(Long customerId, Long addressId);  
  
    /** 获取默认地址 */  
    Address getDefaultAddress(Long customerId);  
}
```

顾客服务 (端口 8083)

- 顾客信息管理
- 收货地址管理
- 内部 Feign 接口供 order-service 调用

```
@Operation(summary = "根据地址ID获取地址信息", description = "根据地址ID获取地址信息 (内部服务调用)")
@GetMapping("/{address/{addressId}")
public Result<Address> getAddressById(
    @Parameter(description = "地址ID", required = true)
    @PathVariable Long addressId) {
    Address address = addressService.getAddressById(addressId);
    if (address == null) {
        return Result.error("地址不存在");
    }
    return Result.success(address);
}
```

```
@Operation(summary = "内部: 根据userId查询顾客", description = "服务间内部调用, 根据 user.id 返回顾客业务主体信息")
@GetMapping("/{internal/user/{userId}")
public Result<Customer> getCustomerById(
    @Parameter(description = "用户ID (user.id)", required = true)
    @PathVariable Long userId) {
    Customer customer = customerService.getCustomerById(userId);
    if (customer == null) {
        return Result.error("顾客信息不存在");
    }
    return Result.success(customer);
}
```

04

Part Four 功能实现-商铺服务

商铺服务 (端口 8084)

- 商品管理

增删改查商品信息

提供分页查询为前端商城提供接口

商户提交商品上架请求后由管理员审核

- 库存管理

- 订单操作

```
public interface ProductService {  
    /** 根据商品ID获取商品信息 ...*/  
    Product getProductById(Long productId);  
  
    /** 根据商户ID获取商品列表 ...*/  
    List<Product> getProductsByShopId(Long shopId);  
  
    /** 搜索商品 ...*/  
    List<Product> searchProducts(String keyword, Long categoryId, Long shopId);  
  
    /** 保存或更新商品信息 ...*/  
    Product saveOrUpdateProduct(Product product);  
  
    /** 删除商品 ...*/  
    boolean deleteProduct(Long productId);  
  
    /** 更新商品状态 ...*/  
    boolean updateProductStatus(Long productId, Integer status);  
  
    /** 获取所有上架商品列表 (用于商城展示) ...*/  
    List<Product> getAllOnSaleProducts(Long categoryId, String keyword);  
  
    /** 分页获取所有上架商品列表 (用于商城展示) ...*/  
    PageResult<Product> getAllOnSaleProductsPage(Long current, Long size, Long categoryId, String keyword);  
  
    /** 分页获取商户商品列表 ...*/  
    PageResult<Product> getProductsByShopIdPage(Long current, Long size, Long shopId, Long categoryId, String keyword);  
  
    /** 分页搜索商品 ...*/  
    PageResult<Product> searchProductsPage(Long current, Long size, String keyword, Long categoryId, Long shopId);  
}
```


商铺服务（端口 8084）

- 商品管理

- 库存管理

商户管理商品的入库操作

在订单发货时选择仓库自动完成出库

下单时原子操作扣减库存，避免超出库存

- 订单操作

```
public interface StockService {  
    /** 根据商品ID获取该商品在所有仓库的库存 ...*/  
    List<WarehouseProduct> getStockByProductId(Long productId);  
  
    /** 根据仓库ID获取该仓库所有商品的库存 ...*/  
    List<WarehouseProduct> getStockByWarehouseId(Long warehouseId);  
  
    /** 获取指定仓库中指定商品的库存 ...*/  
    WarehouseProduct getStock(Long warehouseId, Long productId);  
  
    /** 更新指定仓库中指定商品的库存数量 ...*/  
    WarehouseProduct updateStock(Long warehouseId, Long productId, Integer stock);  
  
    /** 增加指定仓库中指定商品的库存数量 ...*/  
    WarehouseProduct addStock(Long warehouseId, Long productId, Integer stock);  
  
    /** 删除指定仓库中指定商品的库存记录 ...*/  
    boolean deleteStock(Long warehouseId, Long productId);  
  
    /** 扣减指定仓库中指定商品的库存数量（原子操作） ...*/  
    boolean deductStock(Long warehouseId, Long productId, Integer quantity);  
}
```

商铺服务 (端口 8084)

- 商品管理

- 库存管理

- 订单操作

(该部分属于Order服务)

商铺用户在发货时自动调用, 修改订单状态并
创建配送信息

```
public boolean updateOrderStatus(Long orderId, Integer orderStatus) {
    Order order = orderMapper.selectById(orderId);
    if (order == null) {
        throw new BusinessException(ResultCode.FAIL.getCode(), "订单不存在");
    }

    LambdaUpdateWrapper<Order> wrapper = new LambdaUpdateWrapper<>();
    wrapper.eq(Order::getId, orderId).set(Order::getOrderStatus, orderStatus);

    // 订单状态: 0=待支付, 1=待发货, 2=待揽件 (商家已发货、待运输员接单), 3=派送中, 4=已完成, 5=已取消
    if (orderStatus == 2) { // 待揽件 (商户发货)
        // B3: 发货前置校验 - 订单必须是已支付(paymentStatus=1)且处于待发货状态(orderStatus=1)
        if (!Integer.valueOf(1).equals(order.getPaymentStatus())) {
            throw new BusinessException(ResultCode.FAIL.getCode(), "订单未支付, 不能发货");
        }
        if (!Integer.valueOf(1).equals(order.getOrderStatus())) {
            throw new BusinessException(ResultCode.FAIL.getCode(), "订单当前状态不允许发货 (当前状态: " + order.getOrderStatus() + ")");
        }
    }
}
```

订单服务（端口 8085）

- 系统核心业务服务

由用户调用创建，默认状态为待支付/待发货（0/1），由于当前模拟支付，在下单时自动支付

在商铺选择发货时，调用logistics-service自动生成物流路径，同时调用shop-service扣减库存

订单信息内包含商品快照，在创建时存储当时的商品信息，避免商品修改影响历史订单

提供订单回滚，用于在订单取消后库存减扣

```
public interface OrderService extends IService<Order> {

    /** 创建订单 ...*/
    OrderDetailDTO createOrder(Long customerId, CreateOrderRequestDTO request);

    /** 获取订单详情（含订单项） ...*/
    OrderDetailDTO getOrderDetail(Long orderId);

    /** 根据顾客ID获取订单列表 ...*/
    List<Order> getOrdersByCustomerId(Long customerId);

    /** 根据商户ID获取订单列表 ...*/
    List<Order> getOrdersByShopId(Long shopId);

    /** 更新订单状态 ...*/
    boolean updateOrderStatus(Long orderId, Integer orderStatus);

    /** 运输员取消接单后，将订单从「派送中」回到「待揽件」（仅服务间/管理员调用） */
    boolean reopenOrderToPendingPickup(Long orderId);

    /** 取消订单 ...*/
    boolean cancelOrder(Long orderId, String cancelReason);

    /** 支付订单 ...*/
    boolean payOrder(Long orderId);

    /** 顾客软删除订单（仅对顾客隐藏，其他角色仍可见） ...*/
    boolean deleteOrderByCustomer(Long orderId, Long customerId);

    /** 管理员分页查询全部订单（按创建时间倒序），可按状态过滤 ...*/
    PageResult<Order> getAllOrders(Long current, Long size, Integer status);

    /** 根据订单号精确查询订单（管理员用） ...*/
    Order getOrderByNo(String orderNo);
}
```

04

Part Four 功能实现-司机服务

司机服务 (端口 8086)

- 司机与车辆管理
 - 司机个人信息维护
 - 司机名下车辆绑定, 一名司机可绑定多辆车
- 运输任务管理
- 实时位置上报

```
public interface DriverService {  
    /** 根据user_id获取运输员信息 */  
    Driver getDriverByUserId(Long userId);  
  
    /** 根据driver_id获取运输员信息 */  
    Driver getDriverById(Long driverId);  
  
    /** 创建或更新运输员信息 */  
    Driver saveOrUpdateDriver(Driver driver);  
  
    /** 获取所有运输员列表 (管理员使用) */  
    List<Driver> getAllDrivers();  
  
    /** 更新运输员状态 */  
    boolean updateDriverStatus(Long driverId, Integer status);  
}  
  
public interface VehicleService {  
    /** 根据driver_id获取车辆列表 */  
    List<Vehicle> getVehiclesByDriverId(Long driverId);  
  
    /** 根据vehicle_id获取车辆信息 */  
    Vehicle getVehicleById(Long vehicleId);  
  
    /** 创建车辆信息 */  
    Vehicle saveVehicle(Vehicle vehicle);  
  
    /** 更新车辆信息 */  
    Vehicle updateVehicle(Vehicle vehicle);  
  
    /** 删除车辆信息 */  
    boolean deleteVehicle(Long vehicleId);  
  
    /** 更新车辆状态 */  
    boolean updateVehicleStatus(Long vehicleId, Integer status);  
}
```

司机服务（端口 8086）

- 司机与车辆管理
- 运输任务管理
 - 接取待配送订单
 - 完成运输后调用order-service修改订单状态
- 实时位置上报

```
public interface DeliveryService {  
    /** 获取待接单订单列表（分页） */  
    PageResult<OrderDelivery> getPendingDeliveries(Long current, Long size);  
  
    /** 获取运输员的配送订单列表（分页，支持状态筛选） */  
    PageResult<OrderDelivery> getMyDeliveries(Long driverId, Long current, Long size, Integer status, String sortFi  
  
    /** 根据配送ID获取配送详情 */  
    OrderDelivery getDeliveryById(Long deliveryId);  
  
    /** 根据订单ID获取配送信息 */  
    OrderDelivery getDeliveryByOrderId(Long orderId);  
  
    /** 当前运输员进行中的配送（已接单、运输中） */  
    List<OrderDelivery> listInProgressDeliveries(Long driverId);  
  
    /** 接单 */  
    OrderDelivery acceptDelivery(Long deliveryId, Long driverId, Long vehicleId);  
  
    /** 更新配送状态 */  
    OrderDelivery updateDeliveryStatus(Long deliveryId, Long driverId, Integer status, String remark);  
  
    /** 取消配送 */  
    OrderDelivery cancelDelivery(Long deliveryId, Long driverId, String cancelReason);  
  
    /** 创建配送记录（订单服务调用） */  
    OrderDelivery createDelivery(Long orderId, String deliveryAddress, String receiverName, String receiverPhone);  
}
```


04

Part Four 功能实现-司机服务

司机服务 (端口 8086)

- 司机与车辆管理

- 运输任务管理

- 实时位置上报

(当前在logistics-service中)

定期上报GPS位置，更新轨迹信息

```
@Operation(summary = "上报GPS位置", description = "运输员周期性上报位置，同步更新路线当前位置")
@PostMapping("/location")
public Result<LogisticsTrack> uploadLocation(
    @RequestHeader(value = "userId", required = false) String userIdHeader,
    @RequestHeader(value = "roleCode", required = false) String roleCode,
    @RequestBody LocationUpdateDTO locationUpdate) {
    if (!StringUtils.hasText(userIdHeader)) {
        throw new BusinessException(ResultCode.UNAUTHORIZED);
    }
    if (!"driver".equals(roleCode)) {
        throw new BusinessException(ResultCode.FORBIDDEN, "仅运输员可上报位置");
    }
    if (locationUpdate.getRouteId() == null
        || locationUpdate.getLatitude() == null
        || locationUpdate.getLongitude() == null) {
        return Result.error("路线ID、纬度、经度不能为空");
    }

    // B6: userId → driverId 转换 (避免把 userId 直接当 driverId 写入轨迹表)
    Long userId = Long.parseLong(userIdHeader);
    Long driverId;
    try {
        Result<Map<String, Object>> driverResult = driverFeignClient.getDriverByUserId(userId);
        if (driverResult.getCode() != 200 || driverResult.getData() == null) {
            throw new BusinessException(ResultCode.FAIL.getCode(), "运输员信息不存在，请先完善个人资料");
        }
        driverId = Long.valueOf(driverResult.getData().get("id").toString());
    } catch (BusinessException be) {
        throw be;
    } catch (Exception e) {
        throw new BusinessException(ResultCode.FAIL.getCode(), "获取运输员信息失败: " + e.getMessage());
    }

    LogisticsTrack track = trackService.updateLocation(driverId, locationUpdate);
    return Result.success(track);
}
```

路径规划服务（端口 8087）

- 存储物流信息

在创建物流订单时，执行路径规划并将计划路径存储

存储实时上报的位置信息，形成历史路径信息，用于前端展示和数据分析

- 规划物流路径

```
public interface LogisticsTrackService {  
    /* 更新位置 */  
    LogisticsTrack updateLocation(Long driverId, LocationUpdateDTO locationUpdateDTO);  
  
    /* 获取轨迹 */  
    LogisticsTrack getLatestTrack(Long routeId);  
  
    /* 获取轨迹历史 */  
    List<LogisticsTrack> getLatestTracks(Long routeId, Integer limit);  
  
    /* 获取轨迹全部 */  
    List<LogisticsTrack> getTrackByRouteId(Long routeId);  
}
```

04

Part Four 功能实现-路径规划服务

路径规划服务（端口 8087）

- 存储物流信息

- 规划物流路径

使用GraphHopper读取并缓存

OpenStreetMap信息

使用大模型（Deepseek API）结合A*算法提供
规划好的路径

配置文件中routing.strategy来决定选择路径规
划方法

```
// 创建 GraphHopper Bean，读 OSM，用车流规则算路，图数据保存在本地目录。  
;Configuration  
public class GraphHopperConfig {  
  
    private static final Logger log = LoggerFactory.getLogger(GraphHopperConfig.class);  
  
    @Value("${graphhopper.osm-file}")  
    private String osmFile;        // OSM 地图文件路径  
    @Value("${graphhopper.graph-cache}")  
    private String graphCache;     // 图缓存目录（有则加载，无则从 OSM 导入）  
    @Value("${graphhopper.vehicle}")  
    private String vehicle;        // 出行方式，如 car，与 Profile 名一致  
  
    // 初始化引擎：载动车 custom 权重，然后 importOrLoad。  
    @Bean  
    public GraphHopper graphHopper() {  
        log.info("GraphHopperConfig - osmFile: {}", osmFile);  
  
        GraphHopper hopper = new GraphHopper();  
        hopper.setOSMFile(osmFile);  
        hopper.setGraphHopperLocation(graphCache);  
  
        CustomModel carModel = new CustomModel()  
            .setDistanceInfluence(90.0)  
            .addToSpeed(Statement.If( expression: "true", Statement.Op.LIMIT, value: "car_average_speed"))  
            .addToPriority(Statement.If( expression: "!car_access", Statement.Op.MULTIPLY, value: "0"));  
  
        hopper.setProfiles(  
            new Profile(vehicle)  
                .setWeighting("custom")  
                .setCustomModel(carModel)  
        );  
  
        hopper.setEncodedValuesString("car_access, car_average_speed");  
        hopper.importOrLoad();  
  
        log.info("GraphHopperConfig - graphHopper initialized");  
  
        return hopper;  
    }  
}
```


04

Part Four 功能实现-路径规划服务

路径规划服务（端口 8087）

- 存储物流信息

- 规划物流路径

使用GraphHopper读取并缓存

OpenStreetMap信息

使用大模型（Deepseek API）结合A*算法提供
规划好的路径

配置文件中routing.strategy来决定选择路径规
划方法

```
@Configuration
public class RoutingStrategyConfig {

    private static final Logger log = LoggerFactory.getLogger(RoutingStrategyConfig.class);

    @Value("${LLM_WAYPOINT}")
    private String strategyMode;

    /**
     * 声明为 @Primary，确保 RoutePlanningServiceImpl 中的
     * @Autowired RouteStrategy 注入的是此处选定的策略，而非其他实现类
     */
    @Bean
    @Primary
    public RouteStrategy activeRouteStrategy(AStarRouteStrategy astarStrategy,
                                             LlmJudgeRouteStrategy llmJudgeStrategy,
                                             LlmWaypointRouteStrategy llmWaypointStrategy) {
        RouteStrategy selected = switch (strategyMode.toUpperCase()) {
            case "LLM_JUDGE" -> llmJudgeStrategy;
            case "LLM_WAYPOINT" -> llmWaypointStrategy;
            default -> astarStrategy;
        };
        log.info("");
        log.info("  路径规划策略已激活: {} || ", selected.strategyName());
        log.info("");
        return selected;
    }
}
```

前端页面 (端口 8087)

- Pinia 状态管理
利用userStore存储登录信息 (Token)

- Vue Router 路由守卫

- Axios状态管理

```
interface UserInfo {  
  userId: number;  
  username: string;  
  roleCode: string;  
  roleName: string;  
}  
  
export const useUserStore : StoreDefinition<"user", {...}, {...}, {...}> = defineStore( id: 'user', options: {  
  state: () : {token: string, userInfo: ...} => ({  
    token: localStorage.getItem( key: 'token') || null,  
    userInfo: JSON.parse( text: localStorage.getItem( key: 'userInfo') || 'null') as UserInfo | null  
  }),  
  actions: {  
    setUser(token: string, userInfo: UserInfo) : void {  
      this.token = token;  
      this.userInfo = userInfo;  
      localStorage.setItem('token', token);  
      localStorage.setItem('userInfo', JSON.stringify(userInfo));  
    },  
    logout() : void {  
      this.token = null;  
      this.userInfo = null;  
      localStorage.removeItem( key: 'token');  
      localStorage.removeItem( key: 'userInfo');  
    }  
  }  
})
```

前端页面 (端口 8087)

- Pinia 状态管理

- Vue Router 路由守卫

以顾客路由管理为例

用元数据控制角色权限，未登录跳转至登录页面，
其他角色跳转至该角色首页

- Axios状态管理

```
{
  path: '/customer/home',
  component: () => import('@/pages/CustomerHome.vue'),
  meta: {role: 'customer'},
  redirect: '/customer/home/products',
  children: [
    {
      path: 'products',
      component: () => import('@/pages/Customer/Product.vue'),
      meta: {role: 'customer'}
    },
    {
      path: 'orders',
      component: () => import('@/pages/Customer/Order.vue'),
      meta: {role: 'customer'}
    },
    {
      path: 'profile',
      component: () => import('@/pages/Customer/Profile.vue'),
      meta: {role: 'customer'}
    },
    {
      path: 'shop-order',
      component: () => import('@/pages/Customer/CreateOrder.vue'),
      meta: {role: 'customer'}
    },
    {
      path: 'shop/:id',
      component: () => import('@/pages/Customer/ShopPage.vue'),
      meta: {role: 'customer'}
    }
  ]
},
```

前端页面 (端口 8087)

- Pinia 状态管理
- Vue Router 路由守卫
- **Axios状态管理**
封装统一请求拦截, 自动附带 JWT Token

```
const request : AxiosInstance = axios.create({
  baseURL: '/api',
  timeout: 60000
})

request.interceptors.request.use(
  onFulfilled: config : InternalAxiosRequestConfig<any... => {
    const userStore : Store<"user", {token: (string ... = useUserStore();
    if(userStore.token) {
      config.headers.Authorization = `Bearer ${userStore.token}`;
    }
    return config;
  },
  onRejected: error => {
    return Promise.reject(error);
  }
)
```

Part Five

项目亮点



05

Part Five

项目亮点-整体背景

GraphHopper + OpenStreetMap

路网数据来源于 OpenStreetMap，用 GraphHopper 加载解析后构建路网图缓存到 graph-cache` 目录。路网图里每个节点代表一个路口，每条边代表一段道路，边上带有距离属性。

坐标映射

在运行 A* 之前，需要先把用户输入的经纬度坐标映射到路网上最近的节点，这一步通过 GraphHopper 的 `LocationIndex.findClosest()` 实现——它会在路网中找到距离给定坐标最近的有效路网节点，确保起终点都在真实道路上

```
// — 第一步：经纬度 → 最近路网节点 —————  
Snap startSnap = locationIndex.findClosest(startLat, startLon, EdgeFilter.ALL_EDGES);  
Snap endSnap   = locationIndex.findClosest(endLat,   endLon,   EdgeFilter.ALL_EDGES);
```



ADD TEXT



ADD TEXT



ADD TEXT

核心公式:

$$f(n) = g(n) + h(n)$$

$g(n)$ 是从起点沿路网走到当前节点 n 的实际代价, 用已走过的路网边距离累加来表示。

$h(n)$ 是当前节点到终点的启发式估计, 我用的是 Haversine 球面距离公式, 也就是地球表面两点的球面直线距离。

加权A*

在标准 A* 的基础上, 引入启发函数权重 ϵ : $f(n) = g(n) + \epsilon \times h(n)$

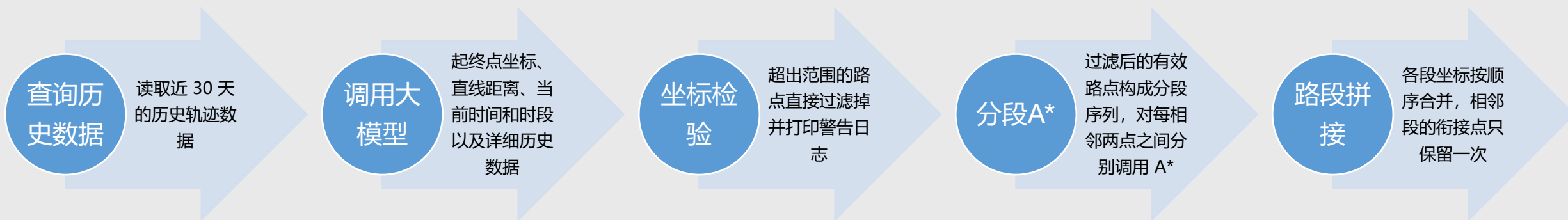
当 $\epsilon = 1.0$ 时, 就是标准最优 A*; 当 $\epsilon > 1.0$ 时, 启发函数被"膨胀", 算法更激进地趋向终点, 会探索出与最优路线不同的备选路径。

```
double h = epsilon * haversine(nodeAccess.getLat(neighbor), nodeAccess.getLon(neighbor),  
                                endLat, endLon);
```

05

Part Five

项目亮点-LLM_WAYPOINT

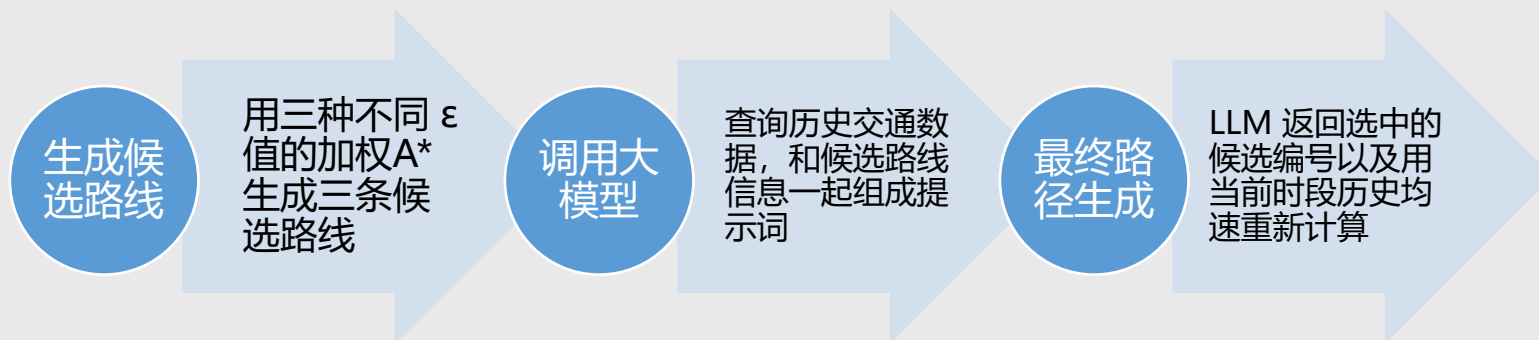


降级机制

如果 LLM 调用失败（网络超时等），或者过滤后没有有效路段，系统会自动降级为纯 A* 直达规划，保证路线一定能生成，用户不会看到报错。

05

Part Five 项目亮点-LLM_JUDGE



策略对比

LLM_WAYPOINT 中 LLM 决定“走哪个方向”，A* 在真实路网上执行每段规划，如果出现 LLM 幻觉坐标在一定程度上会对路线整体质量产生影响；LLM_JUDGE 中 LLM 参与的是评估层，所有候选路线都已经是合法的路网路线，LLM 只是做综合评估选最优的。

当前默认执行 LLM_WAYPOINT 模式

Part Six

功能展示



Part Seven

后续计划





容器化部署

后续计划使用Docker进行容器化部署。计划为每个服务编写 Dockerfile，并通过 Kubernetes 完成容器编排与服务管理。前端使用 Nginx 容器进行托管，并配置反向代理到后端网关。这样整个系统可以通过 Kubernetes 统一部署和管理，具备良好的扩展性与可迁移性。



消息队列

目前服务间都是 Feign 同步调用，一旦某个服务响应慢或挂掉，整个调用链都会阻塞。引入RabbitMQ之后，例如发货这个操作，order-service只需要发一条"订单发货"消息，logistics-service订阅到之后异步去生成路线，两个服务就解耦了。



Redis缓存

商品列表这类数据读多写少，现在每次都查数据库，压力比较大。加了 Redis 之后，第一次查完放进缓存，后续直接从缓存取，能大幅降低数据库压力。另外 Token 黑名单（用户退出登录后让 Token 失效）也计划用 Redis 来实现。



谢谢观看

汇报人：付苗

汇报时间：2026.03.26